

```
In [ ]: Name:Omkar Hulawale
Class-Batch: TE-A3
Roll_No: 13165
```

```
In [16]: import nltk
import string
import numpy as np
import pandas as pd
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
In [17]: # Sample document text
document_text = """
A search algorithm is a method for finding an item or group of items with specific
within a collection of items. Search algorithms are used in databases, search engine
"""
```

```
In [18]: # Tokenization
tokens = word_tokenize(document_text)
```

```
In [19]: # Display results
print("Tokens:", tokens)
```

```
Tokens: ['A', 'search', 'algorithm', 'is', 'a', 'method', 'for', 'finding', 'an', 'i
tem', 'or', 'group', 'of', 'items', 'with', 'specific', 'properties', 'within', 'a',
'collection', 'of', 'items', '.', 'Search', 'algorithms', 'are', 'used', 'in', 'data
bases', ',', 'search', 'engines', ',', 'AI', ',', 'etc', '.']
```

```
In [20]: # POS Tagging
pos_tags = nltk.pos_tag(tokens)
```

```
In [21]: print("POS Tags:", pos_tags)
```

```
POS Tags: [('A', 'DT'), ('search', 'NN'), ('algorithm', 'NN'), ('is', 'VBZ'), ('a',
'DT'), ('method', 'NN'), ('for', 'IN'), ('finding', 'VBG'), ('an', 'DT'), ('item',
'NN'), ('or', 'CC'), ('group', 'NN'), ('of', 'IN'), ('items', 'NNS'), ('with', 'I
N'), ('specific', 'JJ'), ('properties', 'NNS'), ('within', 'IN'), ('a', 'DT'), ('col
lection', 'NN'), ('of', 'IN'), ('items', 'NNS'), ('.', '.'), ('Search', 'NNP'), ('al
gorithms', 'NN'), ('are', 'VBP'), ('used', 'VBN'), ('in', 'IN'), ('databases', 'NN
S'), (',', ','), ('search', 'NN'), ('engines', 'NNS'), (',', ','), ('AI', 'NNP'),
(',', ','), ('etc', 'NN'), ('.', '.')]

```

```
In [22]: # Stop Words Removal
stop_words = set(stopwords.words("english"))
filtered_tokens = [word for word in tokens if word.lower() not in stop_words and wo
```

```
In [23]: print("Filtered Tokens:", filtered_tokens)
```

```
Filtered Tokens: ['search', 'algorithm', 'method', 'finding', 'item', 'group', 'item
s', 'specific', 'properties', 'within', 'collection', 'items', 'Search', 'algorithm
s', 'used', 'databases', 'search', 'engines', 'AI', 'etc']

```

```
In [24]: # Stemming
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in filtered_tokens]
```

```
In [25]: print("Stemmed Words:", stemmed_words)
```

Stemmed Words: ['search', 'algorithm', 'method', 'find', 'item', 'group', 'item', 'specific', 'property', 'within', 'collect', 'item', 'search', 'algorithm', 'use', 'database', 'search', 'engine', 'ai', 'etc']

```
In [26]: # Lemmatization
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(word) for word in filtered_tokens]
```

```
In [27]: print("Lemmatized Words:", lemmatized_words)
```

Lemmatized Words: ['search', 'algorithm', 'method', 'finding', 'item', 'group', 'item', 'specific', 'property', 'within', 'collection', 'item', 'Search', 'algorithm', 'used', 'database', 'search', 'engine', 'AI', 'etc']

```
In [28]: # Convert document to a List (since TfidfVectorizer expects multiple documents)
documents = [document_text]

# Initialize the TF-IDF vectorizer
vectorizer = TfidfVectorizer()

# Fit and transform the document
tfidf_matrix = vectorizer.fit_transform(documents)

# Get the feature names (words)
feature_names = vectorizer.get_feature_names_out()

# Convert TF-IDF matrix to a DataFrame for better readability
df_tfidf = pd.DataFrame(tfidf_matrix.toarray(), columns=feature_names)

# Display the IDF values
idf_values = pd.DataFrame({"Word": feature_names, "IDF": vectorizer.idf_})

print("TF-IDF Matrix:\n", df_tfidf)
print("\nIDF Values:\n", idf_values)
```

TF-IDF Matrix:

	ai	algorithm	algorithms	an	are	collection	databases	\
0	0.160128	0.160128	0.160128	0.160128	0.160128	0.160128	0.160128	
	engines	etc	finding	...	items	method	of	or \
0	0.160128	0.160128	0.160128	...	0.320256	0.160128	0.320256	0.160128
	properties	search	specific	used	with	within		
0	0.160128	0.480384	0.160128	0.160128	0.160128	0.160128		

[1 rows x 25 columns]

IDF Values:

	Word	IDF
0	ai	1.0
1	algorithm	1.0
2	algorithms	1.0
3	an	1.0
4	are	1.0
5	collection	1.0
6	databases	1.0
7	engines	1.0
8	etc	1.0
9	finding	1.0
10	for	1.0
11	group	1.0
12	in	1.0
13	is	1.0
14	item	1.0
15	items	1.0
16	method	1.0
17	of	1.0
18	or	1.0
19	properties	1.0
20	search	1.0
21	specific	1.0
22	used	1.0
23	with	1.0
24	within	1.0